DSA0102 – OBJECT ORIENTED PROGRAMMING IN C++

Name: Sanjay Manesh M

Reg No: 192511362

ASSIGNMENT 3

Q1 – Constructors and Constructor Overloading

Constructors and Constructor Overloading

Constructors are special member functions of a class that are automatically executed when an object is created. They are mainly used to initialize the data members of an object with suitable values. In a library book management system, different books may be registered with different levels of information. Some books may have complete details, donated books may contain only a title, and some records may initially be empty. Constructor overloading allows the Book class to handle these different situations using multiple constructors with different parameter lists.

Default Constructor

A default constructor is a constructor that does not require any arguments. It initializes a newly created object with default values. In the Book class, the default constructor creates an empty book record with an empty title, an unknown author and zero copies. The compiler automatically provides a default constructor only when no constructor has been declared by the programmer. If a parameterised constructor is declared, the compiler does not automatically generate a default constructor, so it must be written explicitly when needed.

Parameterised Constructor and Constructor Overloading

A parameterised constructor accepts values as arguments and initializes the data members of an object. In the library system, it is used to create a book with its title, author and number of copies. Constructor overloading allows multiple constructors with different parameters to exist in the same class. A separate constructor can be used for donated books that contain only a title, while another constructor can initialize all the book details.

Member Initializer List and Constructor Delegation

A member initializer list is used to initialize class data members directly before the constructor body executes. It is preferred over assigning values inside the constructor body because direct initialization avoids unnecessary initialization followed by assignment. Constructor delegation allows one constructor to call another constructor within the same class. In this program, the

default constructor and the title-based constructor delegate their initialization to the parameterised constructor. This reduces code repetition and ensures that all book objects follow the same initialization process.

Order of Member Initialization

In C++, data members are initialized according to the order in which they are declared inside the class, regardless of the order specified in the member initializer list. In the Book class, the title is initialized first, followed by the author and then the number of copies. If the initializer list specifies a different order, the compiler still follows the declaration order and may generate a -Wreorder warning. Maintaining the correct order improves code readability and avoids unexpected initialization problems.

Validation of Number of Copies

The number of copies must be a non-negative integer because a library cannot maintain a negative number of books. A validation function is used inside the parameterised constructor to check the supplied value. If a negative number is provided, the constructor throws an exception and prevents the creation of an invalid Book object. This ensures that every successfully created object contains valid information.

Use of Explicit Constructor

The explicit keyword is used to prevent unwanted implicit conversions when a constructor accepts a single argument. For example, without the explicit keyword, a string value can be automatically converted into a Book object through copy initialization. When the constructor is marked explicit, this implicit conversion is not allowed, although direct initialization remains possible. It is useful when automatic object conversion could cause unexpected results.

Alternative Design and Complexity Analysis

Instead of using multiple overloaded constructors, a single constructor with default arguments can be used to initialize the Book object. For example, the title, author and number of copies can be assigned default values in one constructor. This reduces the number of constructor definitions, but multiple overloaded constructors with constructor delegation provide a clearer representation of the different book registration situations. Both approaches can use the same validation function to check the number of copies.

The time complexity of object creation is O(1), as a fixed number of data members are initialized. The auxiliary space complexity is also O(1), excluding the memory required for the string contents. Constructor delegation helps avoid repeating validation logic in different constructors.

Conclusion

Thus, the Book class demonstrates the use of default, parameterised and overloaded constructors in a library management system. Member initializer lists and constructor delegation provide efficient initialization and reduce code repetition. The validation mechanism prevents negative copy counts, ensuring that invalid book records are not successfully created.

Program Code

```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

class Book
{
private:
    string title;
    string author;
    int copies;

    static int checkCopies(int c)
    {
        if (c < 0)
        {
            throw invalid_argument("Number of copies cannot be negative");
        }
        return c;
    }

public:

    // Default constructor
    Book() : Book("", "Unknown", 0)
    {
    }

    // Constructor for donated books
    explicit Book(string t) : Book(t, "Unknown", 1)
```

```
    {
    }

    // Parameterised constructor
    Book(string t, string a, int c)
        : title(t), author(a), copies(checkCopies(c))
    {
    }

    // Display book details
    void display() const
    {
        cout << "Book Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Number of Copies: " << copies << endl;
        cout << "--------------------------" << endl;
    }
};

int main()
{
    try
    {
        // Creating objects using different constructors
        Book b1;
        Book b2("C++ Programming");
        Book b3("Data Structures", "Mark Allen", 5);

        cout << "DEFAULT CONSTRUCTOR" << endl;
        b1.display();

        cout << "DONATED BOOK CONSTRUCTOR" << endl;
```

```
    b2.display();

    cout << "PARAMETERISED CONSTRUCTOR" << endl;
    b3.display();

    // Array of Book objects using default constructor
    Book shelf[3];

    // Testing negative number of copies
    cout << "TESTING INVALID COPIES" << endl;
    Book b4("Operating Systems", "Galvin", -2);

  }
  catch (const invalid_argument& e)
  {
    cout << "Error: " << e.what() << endl;
  }

  return 0;
}
```

Output

```
DEFAULT CONSTRUCTOR
Book Title:
Author: Unknown
Number of Copies: 0
-------------------------
DONATED BOOK CONSTRUCTOR
Book Title: C++ Programming
Author: Unknown
Number of Copies: 1
-------------------------
PARAMETERISED CONSTRUCTOR
Book Title: Data Structures
Author: Mark Allen
Number of Copies: 5
-------------------------
TESTING INVALID COPIES
Error: Number of copies cannot be negative
```

QUESTION 2 – Copy Constructors: Shallow and Deep Copies

Introduction

A copy constructor is a special constructor in C++ that creates a new object by copying the data of an existing object. It is mainly used when an object is initialized using another object of the same class. Copy constructors are important when a class manages dynamically allocated memory because copying an object incorrectly can cause memory-related problems.

Shallow Copy

A shallow copy copies the values of all data members from one object to another. When a class contains a pointer, only the memory address is copied instead of creating a separate memory allocation. As a result, both objects point to the same memory location. Changes made through one object may affect the other, and deleting the same memory twice can cause a double-free error.

Deep Copy

A deep copy creates a separate memory allocation for the copied object and duplicates the actual data. Each object owns its own memory, so modifying one object does not affect the other. In this program, the Prescription class stores medicine names in a dynamically allocated array. A deep-copy constructor is implemented to allocate new memory and copy each medicine name individually.

Copy Assignment Operator

The copy assignment operator is used when an already existing object is assigned the values of another object. It must release the old memory and allocate new memory for the copied data. A self-assignment check (this != &other) is included to prevent unnecessary operations when an object is assigned to itself.

Destructor and Rule of Three

The destructor releases dynamically allocated memory using delete[]. Without a destructor, dynamically allocated memory may remain occupied even after the object is no longer required. The Rule of Three states that if a class requires a user-defined destructor, copy constructor, or copy assignment operator, it generally needs all three to manage resources correctly.

Situations Where Copy Constructor Is Called

A copy constructor is commonly called when a new object is initialized from an existing object, when an object is passed to a function by value, and when an object is returned from a function by value. Passing an object by constant reference avoids an unnecessary copy because the original object is accessed without creating another object.

Experimental Analysis

The program creates multiple Prescription objects and copies medicine information between them. It modifies the copied objects to verify that the original data remains unchanged. It also checks copy construction, copy assignment, and self-assignment. AddressSanitizer can be used to identify memory errors such as double deletion and invalid memory access.

Alternative Design and Complexity Analysis

Instead of using a dynamically allocated array, the program can use vector<string> to store medicine names. The vector automatically manages memory and provides safe copying, reducing the need to implement a destructor and copy operations manually. Deep copying an array of n medicine names takes O(n) time and O(n) additional space, while a shallow copy only copies the pointer but does not provide independent memory ownership.

Conclusion

Copy constructors and copy assignment operators are essential when a C++ class manages dynamically allocated memory. Deep copying ensures that every object maintains its own independent data and prevents memory-management errors. The Rule of Three helps create reliable classes that manage dynamic resources safely.

Program Code

```
#include <iostream>
#include <string>
#include <chrono>
using namespace std;

class Prescription {
    int n;
    string* medicines;

    static int copiesMade;

public:
    // Parameterized constructor
    explicit Prescription(int count) {
        n = count;
        medicines = new string[n];
```

```
}

// Deep-copy constructor
Prescription(const Prescription& other) {
    n = other.n;
    medicines = new string[n];

    for (int i = 0; i < n; i++) {
        medicines[i] = other.medicines[i];
    }

    copiesMade++;
    cout << "Copy constructor called" << endl;
}

// Copy assignment operator
Prescription& operator=(const Prescription& other) {
    if (this != &other) {
        string* newMedicines = new string[other.n];

        for (int i = 0; i < other.n; i++) {
            newMedicines[i] = other.medicines[i];
        }

        delete[] medicines;

        medicines = newMedicines;
        n = other.n;
    }

    return *this;
}
```

```
    // Destructor
    ~Prescription() {
        delete[] medicines;
    }

    void set(int i, const string& medicine) {
        if (i >= 0 && i < n) {
            medicines[i] = medicine;
        }
    }

    string get(int i) const {
        if (i >= 0 && i < n) {
            return medicines[i];
        }
        return "";
    }

    static void resetCopies() {
        copiesMade = 0;
    }

    static int getCopies() {
        return copiesMade;
    }
};

int Prescription::copiesMade = 0;

// Function receiving object by value
void displayByValue(Prescription p) {
```

```
    cout << "Function received object by value." << endl;
}

// Function receiving object by constant reference
void displayByReference(const Prescription& p) {
    cout << "Function received object by reference." << endl;
}

int main() {
    Prescription p(2);

    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    // Deep copy using copy constructor
    Prescription q = p;

    q.set(1, "Cetirizine");

    // Deep copy using assignment operator
    Prescription r(1);
    r = p;

    p.set(0, "Ibuprofen");

    cout << "\nPrescription details:" << endl;

    cout << "P: " << p.get(0) << " " << p.get(1) << endl;
    cout << "Q: " << q.get(0) << " " << q.get(1) << endl;
    cout << "R: " << r.get(0) << " " << r.get(1) << endl;

    // Self-assignment test
```

```
    p = p;
    cout << "\nSelf-assignment test: " << p.get(0) << endl;

    // Copy count experiment
    Prescription::resetCopies();

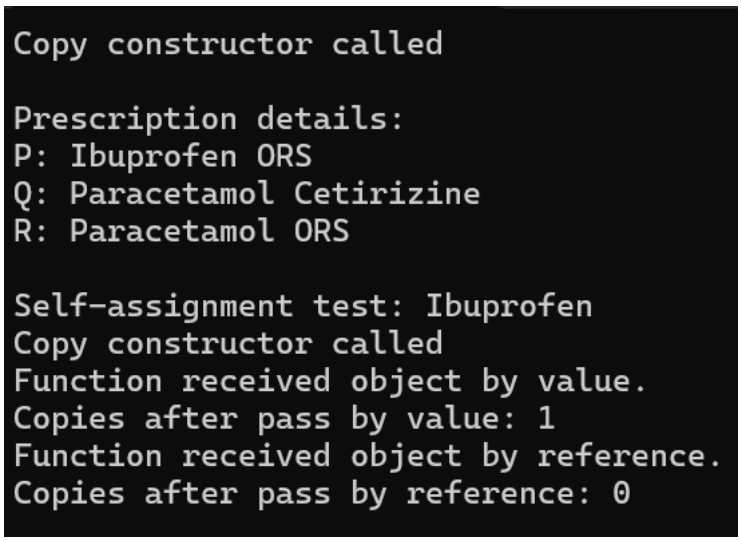
    displayByValue(p);
    cout << "Copies after pass by value: "
         << Prescription::getCopies() << endl;

    Prescription::resetCopies();

    displayByReference(p);
    cout << "Copies after pass by reference: "
         << Prescription::getCopies() << endl;

    return 0;
}
```

Output



QUESTION 3 – Dynamic Constructors and Destructors

Introduction

Constructors and destructors are important features of object-oriented programming in C++. A constructor initializes an object when it is created, while a destructor is automatically called when the object is destroyed. Dynamic memory allocation allows a program to create memory during runtime according to its requirements. In this question, a dynamic two-dimensional seat arrangement is used to represent an auditorium booking system.

Dynamic Memory Allocation

Dynamic memory allocation is performed using the new operator in C++. It allows memory to be allocated during program execution. In an auditorium booking system, the number of rows and seats may vary, so a dynamically allocated two-dimensional array can be used to represent the seating arrangement. Each seat can be marked as booked or available.

Constructor and Destructor

The constructor initializes the auditorium by allocating memory for the required number of rows and seats. The destructor releases the dynamically allocated memory using the delete[] operator. Releasing memory is important because memory that is allocated dynamically is not automatically released when a pointer goes out of scope.

Object Lifetime and Destruction Order

Objects created inside a function or block are automatically destroyed when they go out of scope. Local objects are destroyed in the reverse order of their construction. Objects created dynamically using new remain in memory until they are explicitly destroyed using delete. Therefore, dynamically created objects must be properly deleted to avoid memory leaks.

Booking and Cancelling Seats

The auditorium class provides functions to book and cancel seats. When a seat is booked, its status is changed from available to occupied. When a booking is cancelled, the seat status is changed back to available. The program also checks whether the selected row and seat number are within the valid range.

Importance of Memory Management

The delete[] operator must be used to release memory allocated using new[]. Using delete instead of delete[] for an array can cause undefined behaviour. Similarly, if dynamically allocated memory is not released, the program may experience memory leaks. Proper memory management ensures efficient use of system resources.

Alternative Design and Complexity Analysis

An alternative approach is to allocate all seats in one continuous memory block instead of allocating each row separately. A two-dimensional position can be accessed using the index row * seats + seat. Another option is to use vector<vector<char>>, which automatically

manages memory and reduces the need for manual allocation and deletion. Accessing an individual seat takes O(1) time, while initializing the complete seating arrangement takes O(rows × seats) time.

Conclusion

Dynamic constructors and destructors help manage memory efficiently when the amount of data is decided during program execution. In an auditorium booking system, dynamic memory allocation allows flexible seat arrangements, while the destructor ensures that allocated memory is released properly. Using these concepts helps prevent memory leaks and improves program reliability.

Program Code

```
#include <iostream>

using namespace std;


class Auditorium {

    int rows, seats;

    char** seatMap;


public:

    // Constructor

    Auditorium(int r, int s) {

        rows = r;

        seats = s;


        seatMap = new char*[rows];


        for (int i = 0; i < rows; i++) {

            seatMap[i] = new char[seats];


            for (int j = 0; j < seats; j++) {
```

```
            seatMap[i][j] = 'O';
        }
    }

    cout << "Auditorium created with "
         << rows << " rows and "
         << seats << " seats." << endl;
}

// Book a seat
void bookSeat(int r, int s) {
    if (r < 0 || r >= rows || s < 0 || s >= seats) {
        cout << "Invalid seat position." << endl;
    }
    else if (seatMap[r][s] == 'X') {
        cout << "Seat already booked." << endl;
    }
    else {
        seatMap[r][s] = 'X';
        cout << "Seat booked successfully." << endl;
    }
}

// Cancel a seat
void cancelSeat(int r, int s) {
    if (r < 0 || r >= rows || s < 0 || s >= seats) {
```

```
        cout << "Invalid seat position." << endl;
    }
    else if (seatMap[r][s] == 'O') {
        cout << "Seat is not booked." << endl;
    }
    else {
        seatMap[r][s] = 'O';
        cout << "Booking cancelled successfully." << endl;
    }
}

// Display seats
void display() const {
    cout << "\nSeat Arrangement (O = Available, X = Booked):\n";

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < seats; j++) {
            cout << seatMap[i][j] << " ";
        }
        cout << endl;
    }
}

// Destructor
~Auditorium() {
    for (int i = 0; i < rows; i++) {
```

```
            delete[] seatMap[i];
        }

        delete[] seatMap;

        cout << "Auditorium memory released." << endl;
    }
};

int main() {
    // Stack object
    Auditorium a(3, 4);

    a.bookSeat(0, 1);
    a.bookSeat(1, 2);
    a.display();

    a.cancelSeat(0, 1);
    a.display();

    // Dynamically created object
    Auditorium* b = new Auditorium(2, 3);

    b->bookSeat(0, 0);
    b->display();
```

```
    delete b;

    // Block scope example
    {
        Auditorium c(1, 2);
        c.bookSeat(0, 1);
    }

    cout << "\nEnd of program." << endl;

    return 0;
}
```

Output

```
Auditorium created with 3 rows and 4 seats.
Seat booked successfully.
Seat booked successfully.

Seat Arrangement (O = Available, X = Booked):
O X O O
O O X O
O O O O
Booking cancelled successfully.

Seat Arrangement (O = Available, X = Booked):
O O O O
O O X O
O O O O
Auditorium created with 2 rows and 3 seats.
Seat booked successfully.

Seat Arrangement (O = Available, X = Booked):
X O O
O O O
Auditorium memory released.
Auditorium created with 1 rows and 2 seats.
Seat booked successfully.
Auditorium memory released.

End of program.
Auditorium memory released.
```

QUESTION 4 – Overloading Arithmetic and Stream Operators

Introduction

Operator overloading is a feature of C++ that allows operators to work with user-defined data types. It makes objects behave more naturally when performing arithmetic and comparison operations. In this question, a Money class is designed to represent monetary values and perform arithmetic operations using overloaded operators.

Money Class Representation

The Money class stores the monetary value as an integer number of paise using the long long data type. This avoids floating-point precision problems that can occur when using float or double for monetary calculations. For example, 120 rupees and 75 paise are stored as 12075 paise.

Arithmetic Operator Overloading

The addition operator (+) is overloaded to add the monetary values of two objects and return a new Money object. The subtraction operator (-) subtracts one monetary value from another. The compound addition operator (+=) directly updates the value of the existing object and returns a reference to that object.

Comparison Operator Overloading

The equality operator (==) checks whether two monetary objects have the same value. The less-than operator (<) compares two objects and determines which one has a smaller monetary value. These operators make it easier to compare objects directly without accessing their internal data members.

Stream Insertion and Extraction Operators

The stream insertion operator (<<) is overloaded to display monetary values in a readable format, such as Rs 200.00. The stream extraction operator (>>) allows users to enter monetary values through the keyboard. These operators are generally implemented as non-member friend functions because the left operand is a stream object such as cout or cin.

Member and Friend Functions

Member functions are suitable for operators such as +, -, +=, == and < because they operate on the current object. Friend functions are useful for stream operators because they require access to private data while accepting the stream as the left operand. The expression a + b internally calls the overloaded addition operator.

Alternative Design and Complexity Analysis

An alternative approach is to use named functions such as addMoney() and compareMoney() instead of overloaded operators. However, operator overloading makes expressions shorter and

easier to understand. Each arithmetic or comparison operation takes O(1) time and uses O(1) additional space, since the class stores only one integer value.

Conclusion

Operator overloading improves the readability and usability of user-defined classes. By overloading arithmetic, comparison and stream operators, the Money class can perform monetary calculations in a natural way. Storing values as integer paise also helps maintain accuracy during financial calculations.

Program Code

```
#include <iostream>

#include <iomanip>

using namespace std;



class Money {

    long long paise;



public:

    // Constructor

    Money(long long rupees = 0, int p = 0) {

        paise = rupees * 100 + p;

    }



    // Addition operator

    Money operator+(const Money& other) const {

        Money result;

        result.paise = paise + other.paise;

        return result;

    }
```

```
// Subtraction operator
Money operator-(const Money& other) const {
    Money result;
    result.paise = paise - other.paise;
    return result;
}

// Compound addition operator
Money& operator+=(const Money& other) {
    paise += other.paise;
    return *this;
}

// Equality operator
bool operator==(const Money& other) const {
    return paise == other.paise;
}

// Less-than operator
bool operator<(const Money& other) const {
    return paise < other.paise;
}

// Stream insertion operator
friend ostream& operator<<(ostream& out, const Money& m) {
    long long rupees = m.paise / 100;
```

```
    long long remainingPaise = m.paise % 100;

    if (remainingPaise < 0) {
        remainingPaise = -remainingPaise;
    }

    out << "Rs " << rupees << "."
        << setw(2) << setfill('0') << remainingPaise
        << setfill(' ');

    return out;
}

// Stream extraction operator
friend istream& operator>>(istream& in, Money& m) {
    long long rupees;
    int p;

    cout << "Enter rupees: ";
    in >> rupees;

    cout << "Enter paise: ";
    in >> p;

    if (in && (p < 0 || p >= 100)) {
        in.setstate(ios::failbit);
```

```
            return in;
        }

        m.paise = rupees * 100 + p;
        return in;
    }
};

int main() {
    Money a(120, 75);
    Money b(79, 25);

    Money c = a + b;

    cout << "First Amount: " << a << endl;
    cout << "Second Amount: " << b << endl;

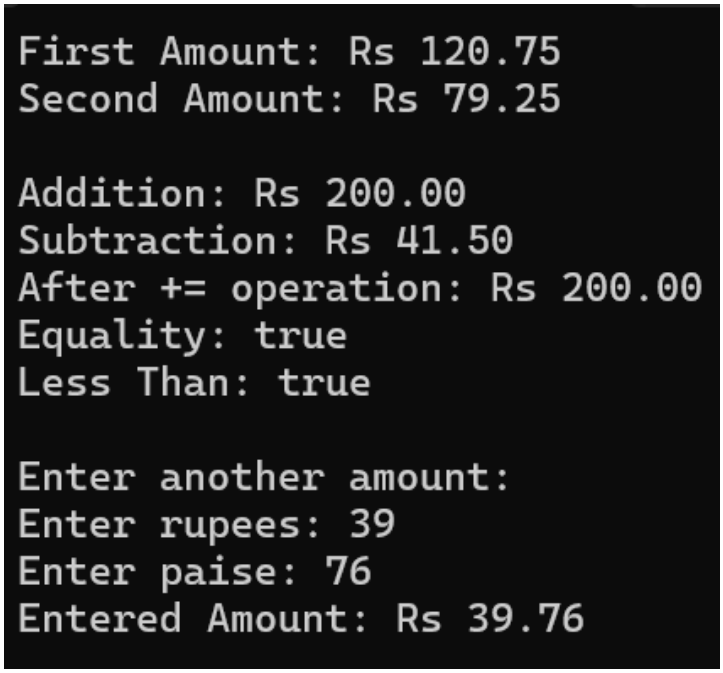
    cout << "\nAddition: " << c << endl;
    cout << "Subtraction: " << a - b << endl;

    a += b;
    cout << "After += operation: " << a << endl;

    cout << boolalpha;
    cout << "Equality: " << (a == c) << endl;
    cout << "Less Than: " << (b < c) << endl;
```

```
    Money d;
    cout << "\nEnter another amount:\n";
    if (cin >> d) {
        cout << "Entered Amount: " << d << endl;
    }
    else {
        cout << "Invalid monetary input." << endl;
    }
    return 0;
}
```

Output



Question 5 – Overloading Increment, Comparison and Subscript Operators

Introduction

Operator overloading in C++ allows operators to perform operations on objects of user-defined classes. It makes programs easier to understand and improves code readability. In this question, a Time class and a Playlist class are designed to demonstrate increment, comparison and subscript operator overloading.

Increment Operator Overloading

The increment operator (++) is overloaded in two forms: prefix and postfix. The prefix increment operator increases the value first and then returns the updated object. The postfix increment operator returns the original value before increasing it. In C++, the postfix version uses an integer dummy parameter to distinguish it from the prefix version.

Prefix and Postfix Increment

Prefix increment is generally more efficient because it directly updates the object and returns a reference. Postfix increment creates a temporary copy of the original object before performing the increment. Therefore, postfix increment may require additional copying operations.

Comparison Operators

The equality operator (==) checks whether two time objects contain the same time value. The less-than operator (<) compares two time objects and determines which one represents an earlier time. These overloaded operators make it possible to compare objects using simple expressions.

Subscript Operator Overloading

The subscript operator ([]) is overloaded in the Playlist class to access individual songs using their index positions. A checked subscript operator verifies whether the requested index is within the valid range. If the index is invalid, the program throws an out_of_range exception instead of accessing invalid memory.

Const and Non-Const Subscript Operators

The non-constant subscript operator returns a reference that allows the user to modify a playlist element. The constant version returns a constant reference and allows only reading. Providing both versions makes the class suitable for both modifiable and read-only objects.

Alternative Design and Complexity Analysis

An alternative approach is to use a normal function such as at(index) to access playlist elements instead of overloading []. The overloaded subscript operator provides a more natural syntax. Prefix and postfix increment operations take O(1) time, while accessing a playlist element using the subscript operator also takes O(1) time. Checked access requires an additional boundary comparison.

Conclusion

Increment, comparison and subscript operator overloading make user-defined classes easier to use. Prefix and postfix operators provide different increment behaviours, comparison operators simplify object comparisons, and the subscript operator allows convenient and safe access to playlist elements.

Program Code

```
#include <iostream>

#include <vector>

#include <string>

#include <stdexcept>

using namespace std;


class Time {

    int minutes;

    int seconds;


    void normalize() {

        if (seconds >= 60) {

            minutes += seconds / 60;

            seconds = seconds % 60;

        }

    }


public:

    Time(int m = 0, int s = 0) {

        minutes = m;

        seconds = s;

        normalize();
```

```
}

// Prefix increment
Time& operator++() {
    seconds++;
    normalize();
    return *this;
}

// Postfix increment
Time operator++(int) {
    Time old = *this;
    seconds++;
    normalize();
    return old;
}

// Equality operator
bool operator==(const Time& other) const {
    return minutes == other.minutes &&
           seconds == other.seconds;
}

// Less-than operator
bool operator<(const Time& other) const {
    if (minutes == other.minutes) {
```

```
            return seconds < other.seconds;
        }
        return minutes < other.minutes;
    }

    void display() const {
        cout << minutes << ":";
        if (seconds < 10) {
            cout << "0";
        }
        cout << seconds << endl;
    }
};

class Playlist {
    vector<string> songs;

public:
    void addSong(const string& song) {
        songs.push_back(song);
    }

    // Non-const subscript operator
    string& operator[](int index) {
        if (index < 0 || index >= (int)songs.size()) {
            throw out_of_range("Invalid playlist index");
```

```
        }
        return songs[index];
    }

    // Const subscript operator
    const string& operator[](int index) const {
        if (index < 0 || index >= (int)songs.size()) {
            throw out_of_range("Invalid playlist index");
        }
        return songs[index];
    }

    int size() const {
        return (int)songs.size();
    }
};

int main() {
    cout << "TIME OPERATIONS" << endl;

    Time t(5, 59);

    cout << "Original Time: ";
    t.display();

    Time a = t++;
```

```
cout << "Postfix Result: ";
a.display();

cout << "After Postfix: ";
t.display();

Time b = ++t;
cout << "Prefix Result: ";
b.display();

cout << boolalpha;
cout << "Times Equal: " << (a == b) << endl;
cout << "First Time Less Than Second: "
     << (a < b) << endl;

cout << "\nPLAYLIST OPERATIONS" << endl;

Playlist p;

p.addSong("Song A");
p.addSong("Song B");
p.addSong("Song C");

cout << "Playlist Songs:" << endl;

for (int i = 0; i < p.size(); i++) {
```

```
        cout << p[i] << endl;
    }

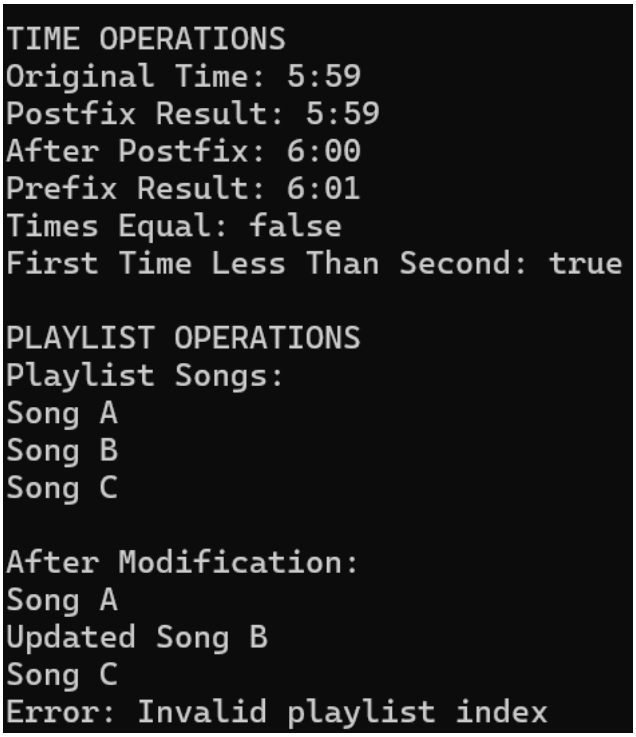
    // Modify a song using subscript operator
    p[1] = "Updated Song B";

    cout << "\nAfter Modification:" << endl;

    for (int i = 0; i < p.size(); i++) {
        cout << p[i] << endl;
    }

    // Invalid index test
    try {
        cout << p[-1] << endl;
    }
    catch (const out_of_range& e) {
        cout << "Error: " << e.what() << endl;
    }
    return 0;
}
```

Output



QUESTION 6 – Rules of Operator Overloading

Introduction

Operator overloading is an important feature of C++ that allows existing operators to be redefined for user-defined data types. It helps objects behave like built-in data types and makes expressions easier to understand. In this question, a `Text` class is implemented to demonstrate operator overloading for string concatenation, equality checking and stream operations.

Text Class and Dynamic Memory

The `Text` class stores a sequence of characters using a dynamically allocated character array. The constructor allocates memory for the given text, and the destructor releases the allocated memory. A copy constructor is implemented to create an independent copy of an existing object. This avoids multiple objects sharing the same memory address.

Overloading the Addition Operator

The addition operator (+) is overloaded to combine two text objects into a new text object. It creates a new character array large enough to store both strings and copies their contents into it. This allows expressions such as `a + b` to concatenate two text objects.

Equality and Inequality Operators

The equality operator (==) checks whether two text objects contain the same characters. The inequality operator (!=) checks whether their contents are different. These operators make it easier to compare text objects without manually comparing individual characters.

Stream Operator Overloading

The stream insertion operator (<<) is overloaded to display the contents of a text object using `cout`. It is implemented as a friend function so that it can access the private character array. This allows text objects to be printed directly.

Rules of Operator Overloading

C++ provides certain rules for operator overloading. New operators cannot be created, and the precedence and associativity of existing operators cannot be changed. At least one operand must be a user-defined type. Some operators, such as scope resolution (`::`), member access (`.`), conditional (`?:`) and `sizeof`, cannot be overloaded. The assignment operator (=) must be overloaded as a member function when a custom assignment implementation is required.

Self-Assignment and Memory Management

Self-assignment occurs when an object is assigned to itself, such as `a = a`. The assignment operator must check for this situation to avoid deleting the object's existing data before copying it. Proper memory management is necessary to prevent memory leaks, invalid memory access and double deletion.

Alternative Design and Complexity Analysis

An alternative is to use the built-in `string` class from the C++ Standard Library instead of managing character arrays manually. The `string` class automatically manages memory and provides built-in concatenation and comparison operations. For a custom text class, concatenating strings of lengths `n` and `m` takes O(n + m) time and requires O(n + m) space for the resulting string.

Conclusion

Operator overloading allows user-defined classes to support familiar operations such as addition and comparison. Implementing copy constructors, assignment operators and destructors ensures safe memory management when dynamic arrays are used. These concepts help develop reliable and reusable C++ classes.

Program Code

```
#include <iostream>

#include <cstring>

using namespace std;
```

```
class Text {
    char* data;

public:
    // Constructor
    Text(const char* str = "") {
        data = new char[strlen(str) + 1];
        strcpy(data, str);
    }

    // Deep-copy constructor
    Text(const Text& other) {
        data = new char[strlen(other.data) + 1];
        strcpy(data, other.data);
    }

    // Copy assignment operator
    Text& operator=(const Text& other) {
        if (this != &other) {
            char* newData = new char[strlen(other.data) + 1];
            strcpy(newData, other.data);

            delete[] data;
            data = newData;
        }
```

```
    return *this;
}

// Addition operator
Text operator+(const Text& other) const {
    int length = strlen(data) + strlen(other.data);

    char* temp = new char[length + 1];

    strcpy(temp, data);
    strcat(temp, other.data);

    Text result(temp);

    delete[] temp;

    return result;
}

// Equality operator
bool operator==(const Text& other) const {
    return strcmp(data, other.data) == 0;
}

// Inequality operator
bool operator!=(const Text& other) const {
```

```
        return strcmp(data, other.data) != 0;
    }

    // Stream insertion operator
    friend ostream& operator<<(ostream& out, const Text& t) {
        out << t.data;
        return out;
    }

    // Destructor
    ~Text() {
        delete[] data;
    }
};

int main() {
    Text a("Hello ");
    Text b("World");

    // Concatenation
    Text c = a + b;

    cout << "First Text: " << a << endl;
    cout << "Second Text: " << b << endl;
    cout << "Combined Text: " << c << endl;
```

```
    // Equality comparison
    Text d("Hello ");

    cout << boolalpha;
    cout << "a == d: " << (a == d) << endl;
    cout << "a != b: " << (a != b) << endl;

    // Copy constructor test
    Text e = c;
    cout << "Copied Text: " << e << endl;

    // Assignment operator test
    Text f("Temporary");
    f = c;

    cout << "Assigned Text: " << f << endl;

    // Self-assignment test
    f = f;

    cout << "After Self Assignment: " << f << endl;

    return 0;
}
```

Output

```
First Text: Hello
Second Text: World
Combined Text: Hello World
a == d: true
a != b: true
Copied Text: Hello World
Assigned Text: Hello World
After Self Assignment: Hello World
```

QUESTION 7 – Design Complex Number Class

Introduction

A complex number is a number that consists of two parts: a real part and an imaginary part. It is generally represented in the form \(a + bi\), where a represents the real part, b represents the imaginary part, and i represents the imaginary unit. Complex numbers are widely used in electrical engineering, especially in the analysis of AC circuits. In this question, a Complex class is designed to perform mathematical operations and calculate impedance in series and parallel RLC circuits.

Complex Number Class

The Complex class stores two data members: the real part and the imaginary part. Constructors are used to initialize complex number objects. A default constructor initializes both parts to zero, a single-parameter constructor initializes the real part, and a parameterized constructor initializes both real and imaginary parts.

Arithmetic Operator Overloading

The addition operator (+) is overloaded to add two complex numbers by adding their real and imaginary parts separately. The subtraction operator (-) subtracts the corresponding parts. The multiplication operator (*) follows the mathematical formula for complex multiplication. The division operator (/) performs complex division and checks whether the denominator is zero before carrying out the operation.

Comparison and Stream Operators

The equality operator (==) checks whether two complex numbers have equal real and imaginary parts. The stream insertion operator (<<) displays complex numbers in a readable format, such as 10 + 5i. These overloaded operators make calculations easier and improve the readability of the program.

Mathematical Operations on Complex Numbers

For two complex numbers $a+bi$ and $c+di$, addition is calculated as $(a+c)+(b+d)i$, while subtraction is calculated as $(a-c)+(b-d)i$. Multiplication is calculated as $(ac-bd)+(ad+bc)i$. Division is performed by multiplying the numerator and denominator by the conjugate of the denominator.

The magnitude of a complex number is calculated using:

$$ |z|=\sqrt{a^2+b^2} $$

Application in AC Circuit Analysis

Complex numbers are useful in AC circuit analysis because they represent impedance, which contains both resistance and reactance. The impedance of a resistor is represented by a real value, while the impedance of an inductor and capacitor contains imaginary values.

The impedance of an inductor is calculated using $Z_L=j\omega L$, and the impedance of a capacitor is calculated using $Z_C=-j/(\omega C)$, where $\omega=2\pi f$ represents angular frequency.

In a series RLC circuit, the total impedance is the sum of the individual impedances. In a parallel RLC circuit, the total impedance is calculated using the reciprocal of the sum of the individual reciprocal impedances.

Alternative Design and Complexity Analysis

An alternative approach is to use the built-in complex<double> class available in the C++ Standard Library. It provides predefined arithmetic operations and mathematical functions for complex numbers. However, implementing a custom class helps understand operator overloading, constructors and complex number calculations. Each arithmetic operation requires O(1) time and O(1) additional space because only two numerical values are stored.

Conclusion

The complex number class demonstrates the practical use of constructors and operator overloading in C++. It simplifies mathematical calculations and provides an effective way to represent electrical impedance. Applying complex numbers to series and parallel RLC circuits shows how object-oriented programming can be used to solve engineering problems.

Program Code

```
#include <iostream>

#include <cmath>

#include <stdexcept>

using namespace std;
```

```
class Complex {
    double real, imag;

public:
    // Default constructor
    Complex() {
        real = 0;
        imag = 0;
    }

    // Real-only constructor
    Complex(double r) {
        real = r;
        imag = 0;
    }

    // Parameterized constructor
    Complex(double r, double i) {
        real = r;
        imag = i;
    }

    // Addition operator
    Complex operator+(const Complex& c) const {
        return Complex(real + c.real, imag + c.imag);
    }
```

```
// Subtraction operator
Complex operator-(const Complex& c) const {
    return Complex(real - c.real, imag - c.imag);
}

// Multiplication operator
Complex operator*(const Complex& c) const {
    return Complex(
        real * c.real - imag * c.imag,
        real * c.imag + imag * c.real
    );
}

// Division operator
Complex operator/(const Complex& c) const {
    double denominator =
        c.real * c.real + c.imag * c.imag;

    if (denominator == 0) {
        throw runtime_error("Division by zero complex number");
    }

    return Complex(
        (real * c.real + imag * c.imag) / denominator,
        (imag * c.real - real * c.imag) / denominator
```

```
    );
}

// Equality operator
bool operator==(const Complex& c) const {
    const double epsilon = 1e-9;

    return fabs(real - c.real) < epsilon &&
           fabs(imag - c.imag) < epsilon;
}

// Magnitude calculation
double magnitude() const {
    return sqrt(real * real + imag * imag);
}

// Stream insertion operator
friend ostream& operator<<(ostream& out, const Complex& c) {
    out << c.real;

    if (c.imag >= 0) {
        out << " + " << c.imag << "i";
    }
    else {
        out << " - " << -c.imag << "i";
    }
```

```
        return out;
    }
};

int main() {
    try {
        // Complex number operations
        Complex a(5, 3);
        Complex b(2, 4);

        cout << "COMPLEX NUMBER OPERATIONS" << endl;

        cout << "First Number: " << a << endl;
        cout << "Second Number: " << b << endl;

        cout << "Addition: " << a + b << endl;
        cout << "Subtraction: " << a - b << endl;
        cout << "Multiplication: " << a * b << endl;
        cout << "Division: " << a / b << endl;

        cout << "Magnitude: " << a.magnitude() << endl;

        cout << boolalpha;
        cout << "Equality: " << (a == b) << endl;
```

```
// RLC circuit parameters
double R = 10.0;
double L = 0.1;
double C = 0.001;
double f = 50.0;

const double PI = acos(-1.0);
double omega = 2 * PI * f;

// Individual impedances
Complex ZR(R, 0);
Complex ZL(0, omega * L);
Complex ZC(0, -1.0 / (omega * C));

cout << "\nRLC CIRCUIT ANALYSIS" << endl;

cout << "Resistance Impedance: " << ZR << endl;
cout << "Inductor Impedance: " << ZL << endl;
cout << "Capacitor Impedance: " << ZC << endl;

// Series impedance
Complex Zseries = ZR + ZL + ZC;

cout << "\nSeries Impedance: "
     << Zseries << endl;
```

```
        cout << "Series Impedance Magnitude: "
             << Zseries.magnitude() << " Ohms" << endl;

        // Parallel impedance
        Complex one(1, 0);

        Complex Zparallel = one /
            (one / ZR + one / ZL + one / ZC);

        cout << "\nParallel Impedance: "
             << Zparallel << endl;

        cout << "Parallel Impedance Magnitude: "
             << Zparallel.magnitude() << " Ohms" << endl;
    }
    catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }

    return 0;
}
```

Output

```
COMPLEX NUMBER OPERATIONS
First Number: 5 + 3i
Second Number: 2 + 4i
Addition: 7 + 7i
Subtraction: 3 - 1i
Multiplication: -2 + 26i
Division: 1.1 - 0.7i
Magnitude: 5.83095
Equality: false

RLC CIRCUIT ANALYSIS
Resistance Impedance: 10 + 0i
Inductor Impedance: 0 + 31.4159i
Capacitor Impedance: 0 - 3.1831i

Series Impedance: 10 + 28.2328i
Series Impedance Magnitude: 29.9515 Ohms

Parallel Impedance: 1.11471 - 3.14715i
Parallel Impedance Magnitude: 3.33873 Ohms
```